CLAUDE.md — LAY:D 프로젝트 AI 협업 가이드

> 이 문서는 Claude가 LAY:D 프로젝트에서 일관된 방식으로 코드를 작성하고 수정하도록 안내한다.
> 모든 코드 작업 전에 반드시 읽어야 한다.

---

1. 절대 규칙 (Never Violate)

```
❌ FastAPI / 백엔드 코드 절대 작성 금지
❌ CSS 파일 생성 금지 — 모든 스타일은 inline JS object
❌ 외부 UI 라이브러리 import 금지 (MUI, Chakra, Tailwind 등)
❌ 실제 DB 연결 코드 작성 금지
✅ 모든 데이터는 Mock (src/mock/*.js 또는 컴포넌트 내 상수)
✅ React 18 hooks만 사용 (useState, useEffect, useRef, useMemo, useCallback)
```

---

2. 코드 스타일

2.1 스타일 작성 방식

```jsx
// ✅ 올바른 방식 — inline style object
const PANEL = {
  background: "rgba(200,200,215,0.07)",
  border: "1px solid rgba(255,255,255,0.1)",
  borderRadius: "12px",
};
```

```
// ✅ 컴포넌트 내 인라인
<div style={{ display: "flex", gap: "10px" }}>

// ❌ 금지
import styles from './Component.module.css';
import styled from 'styled-components';
```

2.2 공통 스타일 상수 (ParameterTab.jsx)

```js
const iS  = { /* 작은 input */ }
const iF  = { /* 전체너비 input */ }
const sS  = { /* 작은 select */ }
const sF  = { /* 전체너비 select */ }
const bS  = { /* 일반 버튼 */ }
const bI  = { /* 아이콘 버튼 */ }
const SEC = { /* 섹션 레이블 (9px uppercase) */ }
const PANEL = { /* 패널 컨테이너 */ }
```

새 스타일이 필요하면 이 상수들을 spread해서 확장한다.

2.3 색상 상수

```js
const EMERALD = "#10b981"  // 성공/shrink ratio
const CYAN    = "#38bdf8"  // 선택/polygon
const PURPLE  = "#a78bfa"  // serif
const ORANGE  = "#fb923c"  // hole
const YELLOW  = "#facc15"  // anchor dot / ruler
```

---

3. 좌표계 — 절대 혼동 금지

3.1 Y-up 좌표계
LAY:D의 모든 world 좌표는 **Y-up** (수학적 좌표계)를 사용한다.
Canvas는 Y-down이므로 변환이 필요하다.

```
World (Y-up)      Canvas (Y-down)
   Y↑              ┌──→ X
    │              │
    └──→ X           ↓ Y
```

3.2 변환 함수 (toC / toW)
```js
// PCell 중심이 canvas 중앙에 오도록
const vpCX = coordX + pcW/2;
const vpCY = coordY + pcH/2;

const toC = (wx, wy) => ({
  cx: CW/2 + (wx - vpCX) * sc,
  cy: CH/2 - (wy - vpCY) * sc,  // ← 부호 반전 핵심
});
const toW = (cx, cy) => ({
  wx: (cx - CW/2) / sc + vpCX,
  wy: -(cy - CH/2) / sc + vpCY, // ← 부호 반전 핵심
});
```

3.3 PCell 좌표 규칙
```
```

```
anchor point = PCell 좌하단 = (coordX, coordY)
PCell local  = [0, pcW] × [0, pcH]  (Y-up)
World coords = PCell-local + (coordX, coordY)
viewport center = (coordX + pcW/2, coordY + pcH/2)
```

3.4 gao() 함수 — 정확한 사용법

```js
// container [0,pw]×[0,ph] 내에서 (ow×oh)의 bottom-left 위치 반환
// "top" = ph 쪽(높은 y), "bottom" = 0 쪽(낮은 y)
function gao(align, pw, ph, ow, oh) {
  const bx = align.includes("left")   ? 0
           : align.includes("right")  ? pw - ow
           : (pw - ow) / 2;
  const by = align.includes("bottom") ? 0
           : align.includes("top")    ? ph - oh
           : (ph - oh) / 2;
  return [bx, by];
}

// ✅ 올바른 사용: paWorld bbox 내에서 hole group 위치
const [hax, hay] = gao(h.align, paWorld.w, paWorld.h, tW, tH);
const sX = paWorld.ox + hax;   // NOT paWorld.ox + paWorld.w/2 +
hax
const sY = paWorld.oy + hay;

// ✅ hole row 방향: row=0이 시각적 최상단 (Y-up에서 가장 큰 y)
const holeOy = sY + (M - 1 - row) * (hh + gY) + gY;
```

3.5 Rect 렌더링 (Y-up → canvas)

```js
```

```
// o.oy = bbox bottom-left y (Y-up)
// o.oy + o.h = bbox top y (Y-up) → canvas에서 위쪽
const {cx, cy} = toC(o.ox, o.oy + o.h);  // canvas top-left
ctx.fillRect(cx, cy, o.w * sc, -o.h * sc); // 음수 높이로 아래→위
```

---

4. ParameterTab 컴포넌트 수정 가이드

4.1 computeRendered() 수정 시
- `pts` → PCell-local Y-up 좌표
- `basePts` → gao 적용 후 PCell-local
- `oPts` → `basePts + algDelta + (coordX, coordY)` = world 좌표
- `paWorld` → world 좌표 bbox `{ox, oy, w, h}`
- Serif/Hole은 반드시 `paWorld` 기준으로 계산

4.2 새 알고리즘 추가 시
```js
// MOCK_FNS에 추가
const MOCK_FNS = [
  { name: "field" },
  { name: "새함수" },  // ← 추가
];

// applyAlgs() 내부에 계산 로직 추가
if (a.fn === "새함수") {
  delta = r6(/* 계산식 */ * f);
}
```

4.3 Canvas draw 수정 시

- draw useEffect dependency array에 새 state 추가 필수
- 항상 흰색 배경 (`ctx.fillStyle="#ffffff"`) 유지
- Y-up 좌표계로 그림 → `toC()` 사용 필수
- `ctx.setLineDash([])` 로 점선 초기화 잊지 말 것

4.4 Dialog 추가/수정 시
- zIndex 계층: 일반 Dialog=700, 중첩 Dialog=800, Preview 내 Dialog=900
- 모든 Dialog는 `position:fixed, inset:0` 오버레이
- onSave 후 반드시 dialog state를 null로 초기화

---

5. 데이터 흐름 패턴

5.1 State 위치
```
ParameterTab (top)
├── pcells[]          ← 모든 PCell 데이터
├── selPcellId        ← 현재 선택된 PCell
├── selPolyId         ← 현재 선택된 Polygon
├── coordX, coordY    ← 시뮬레이션 좌표
└── pcellDlg, polyDlg ← Dialog 열림 상태

PreviewCanvas (child)
├── imode             ← S/H/E/⇧D 현재 모드
├── rulers[]          ← 측정된 ruler 목록
└── serifDlg, holeDlg ← Serif/Hole Dialog 열림 상태
```

5.2 Polygon 업데이트 패턴
```js
```

```
// PreviewCanvas에서 Serif/Hole 변경 시
const updatePolygon = useCallback(poly => {
  setPcells(ps => ps.map(p =>
    p.id === selPcellId
      ? { ...p, polygons: p.polygons.map(pg => pg.id === poly.id ? poly
: pg) }
      : p
  ));
}, [selPcellId]);

// PreviewCanvas로 전달
<PreviewCanvas onUpdatePolygon={updatePolygon} />
```

---

6. Preview Canvas 인터랙션 규칙

6.1 모드 키 매핑

키	모드	동작
`S`	serif-add	canvas 클릭 → SerifDialog → 저장 후 모드 해제
`H`	hole-add	canvas 클릭 → HoleDialog → 저장 후 모드 해제
`E`	edit	객체 클릭 → 해당 Dialog → 저장 후 모드 해제
`⇧D`	delete	객체 클릭 → 즉시 삭제 → 모드 해제
`Esc`	—	모드 해제, ruler 초기화

6.2 모드 해제 패턴

```js
// 액션 완료 즉시 모드 해제
imodeRef.current = null;
setImode(null);
```

```

### 6.3 imodeRef가 필요한 이유
`imode` state는 비동기(React re-render)라서 onClick 핸들러에서 stale 값을 읽을 수 있다.
`imodeRef.current`는 동기적으로 항상 최신값을 읽는다. **항상 둘 다 업데이트**해야 한다.
```js
imodeRef.current = "serif-add";
setImode("serif-add");
```

---

7. FieldMap 규칙

```js
// fieldMap.js를 통해 컬럼명 ↔ 표시명 변환
import { toLabel, toColumn } from '../config/fieldMap.js';

// API → UI
const displayName = toLabel('layout', 'actv_dum_col'); // "Active Dummy Col"

// UI → API
const columnName = toColumn('layout', 'Active Dummy Col'); // "actv_dum_col"
```

---

8. 자주 하는 실수 체크리스트
```

코드를 수정하기 전에 확인:

- [ ] `gao()` 호출 시 container로 `paWorld.w/h` 넘기고, 결과에 `paWorld.ox/oy`만 더하는가? (`.w/2` 중복 가산 금지)
- [ ] hole row 방향: `(M-1-row)` 로 Y-up top-to-bottom 계산하는가?
- [ ] `toC()` 에서 `-(wy - vpCY)` 부호가 맞는가?
- [ ] rect 렌더링에서 `toC(o.ox, o.oy + o.h)` 로 top 좌표를 쓰는가?
- [ ] draw useEffect의 dependency array가 완전한가?
- [ ] Dialog에서 onSave 후 dialog state null 초기화했는가?
- [ ] 모드 완료 후 `imodeRef.current=null; setImode(null)` 둘 다 했는가?
- [ ] 새 스타일에 CSS 파일이나 외부 라이브러리를 쓰지 않는가?
- [ ] FastAPI 코드를 작성하지 않았는가?

---

9. 파일 수정 시 주의사항

ParameterTab.jsx 수정 시
- 844줄 이상의 파일 — 전체 재작성 대신 **python str.replace 방식** 선호
- 전체 재작성이 필요한 경우 bash heredoc 사용 (`cat > file << 'EOF'`)
- 수정 후 반드시 검증: `python3 -c "content=open('ParameterTab.jsx').read(); print(검증식)"`

Mock 데이터 수정 시
- `src/mock/` 하위 파일만 수정
- FastAPI 교체 시 이 파일들을 API 호출로 대체

---

10. 향후 FastAPI 전환 시

현재 mock 위치 → 교체 대상:
```
MOCK_LAYERS     → GET /api/layers?product_id={id}
MOCK_FNS        → GET /api/algorithms/functions
productDB       → GET /api/products
shrink_ratio    → GET /api/products/{id}/shrink_ratios
ParameterTab    → GET/POST /api/layouts/{id}/parameters
```

교체 원칙:
1. fieldMap.js의 apiToUi/uiToApi 변환 함수 사용
2. 로딩 상태(useState loading) 추가
3. 에러 처리(try/catch) 추가
4. Mock 데이터 파일은 삭제하지 말고 `_mock_` prefix로 보존